

Flood Detection Pipeline Documentation

MODIS and SAR Satellite Data Processing

Rio Grande Region Flood Analysis

Document Version: 1.0

Date: January 2025

Project: Flood Detection and Monitoring

Region: Rio Grande, South America

Table of Contents

1. Overview
2. MODIS Pipeline (MODIS_RioGrande_v1.ipynb)
3. SAR Pipeline (SAR_RioGrande_v2.ipynb)
4. Output Integration
5. Common Components

Document Overview

This document provides a comprehensive technical overview of the flood detection pipelines for both MODIS (Moderate Resolution Imaging Spectroradiometer) and SAR (Synthetic Aperture Radar) satellite data processing. Both pipelines process the same Earth Engine asset (shapefile) and can be integrated for complementary flood analysis.

Key Features: - Complete pipeline documentation with code examples - Detailed explanation of spatial aggregation methods - Step-by-step integration guide for combining MODIS and SAR outputs - Configuration and setup instructions - Best practices and recommendations

Overview

Both pipelines are designed to process satellite imagery for flood detection in the Rio Grande region. They share:

- **Common Earth Engine Asset:** Both use the same shapefile FeatureCollection
- **Same Polygon Grid:** Both process polygons from the same spatial grid
- **Spatial Aggregation:** Both aggregate pixel values within polygon boundaries
- **Parallel Processing:** Both use multi-threading for efficient processing
- **Configuration-Based:** Both use YAML configuration files

Key Differences

Aspect	MODIS	SAR
Sensor Type	Optical (Terra & Aqua)	Radar (Sentinel-1)
Resolution	250m (pan-sharpened)	~10m
Cloud Sensitivity	Affected by clouds	All-weather
Temporal Frequency	Daily	6-12 days
Water Detection	Spectral indices	Backscatter thresholds

MODIS Pipeline

Purpose

The MODIS pipeline processes optical satellite imagery from MODIS Terra and Aqua satellites to detect water and calculate spectral statistics for each polygon in the study area.

Pipeline Architecture

1. Initialization & Configuration
 - Load YAML config
 - Initialize Earth Engine

- Setup logging

↓

2. Environment Setup

- Load shapefile FeatureCollection
- Filter by bounding box
- Extract polygon IDs

↓

3. For Each Polygon (Parallel Processing)

Get MODIS Terra & Aqua collections
 Join GQ (250m) and GA (500m) products
 Pan-sharpen 500m bands to 250m
 Calculate spectral indices
 Extract QA bands
 Water detection (3 threshold variants)
 Calculate statistics (mean + percentiles)

↓

4. Output Generation

- Merge statistics from all collections
- Save to CSV with timestamp
- Track failed polygon IDs

Detailed Processing Steps

1. Data Collection The pipeline collects MODIS data from two products:

GQ Product (250m resolution):

```
def dfo_bands_gq(collection):
    return collection.select(
        ["sur_refl_b01", "sur_refl_b02", "num_observations"],
        ["red_250m", "nir_250m", "obs250"],
    )
```

GA Product (500m resolution):

```
def dfo_bands_ga(collection):
    return collection.select(
        [
            "sur_refl_b01", "sur_refl_b02", "sur_refl_b03",
            "sur_refl_b04", "sur_refl_b05", "sur_refl_b06", "sur_refl_b07",
            "state_1km", "num_observations_500m",
        ],
    )
```

```

        "red_500m", "nir_500m", "blue", "green",
        "band5", "band6", "swir", "state_1km", "obs500",
    ],
)

```

Collection Joining:

```

def join_collections(img_coll1, img_coll2):
    filter_time_eq = ee.Filter.equals(
        leftField="system:time_start", rightField="system:time_start"
    )
    joined = ee.Join.inner().apply(img_coll1, img_coll2, filter_time_eq)

    def image_cat(image):
        return ee.Image.cat(image.get("primary"), image.get("secondary"))

    return joined.map(image_cat)

```

2. Pan-Sharpening The 500m bands are pan-sharpened to 250m resolution using a ratio-based method:

```

def pan_sharpen(image):
    red_250m = image.select("red_250m")
    red_250m_safe = red_250m.where(red_250m.eq(0), 0.0000001)
    ratio = image.select("red_500m").divide(red_250m_safe)

    # Apply ratio to all 500m bands
    blue_ps = image.select("blue").divide(ratio)
    swir_ps = image.select("swir").divide(ratio)
    green_ps = image.select("green").divide(ratio)
    nir_500m_ps = image.select("nir_500m").divide(ratio)
    band5_ps = image.select("band5").divide(ratio)
    band6_ps = image.select("band6").divide(ratio)

    return (
        image.select("red_250m", "nir_250m", "obs250", "state_1km")
        .addBands([blue_ps, green_ps, swir_ps, nir_500m_ps, band5_ps, band6_ps, obs500_ps])
        .set({"ratio_scale": ratio.projection().nominalScale()})
    )

```

How it works: - Calculates a ratio between 500m red band and 250m red band - Applies this ratio to all 500m bands to downscale them to 250m - Preserves spectral relationships while increasing spatial resolution

3. Spectral Index Calculation A key ratio for water detection is calculated:

```

def b1b2_ratio(img):
    exp = "float(b('nir_250m') + 13.5) / float(b('red_250m') + 1081.1)"

```

```
dfo_ratio = img.expression(exp)
return img.addBands(dfo_ratio.select([0], ["b1b2_ratio"])).copyProperties(img)
```

This ratio exploits the fact that water has low near-infrared reflectance relative to red reflectance.

4. Quality Assurance (QA) Band Extraction MODIS includes quality flags in the `state_1km` band. The pipeline extracts:

```
def get_qa_bits(image, start, end, new_name):
    pattern = 0
    for i in range(start, end + 1):
        pattern += pow(2, i)
    return image.select([0], [new_name]).bitwiseAnd(pattern).rightShift(start)

def add_qa_bands(img):
    qa_band = img.select("state_1km")
    cloud_state = get_qa_bits(qa_band, 0, 1, "cloud_state")
    cloud_shadow = get_qa_bits(qa_band, 2, 2, "cloud_shadow")
    ice_flag = get_qa_bits(qa_band, 12, 12, "ice_flag")
    snow_flag = get_qa_bits(qa_band, 15, 15, "snow_flag")
    return img.addBands([cloud_state, cloud_shadow, ice_flag, snow_flag])
```

QA Flags Extracted: - `cloud_state`: 0=clear, 1=cloudy, 2=mixed, 3=not set - `cloud_shadow`: 0=no, 1=yes - `ice_flag`: 0=no, 1=yes - `snow_flag`: 0=no snow, 1=snow

5. Water Detection Water is detected using a multi-threshold approach:

```
def water_detection(modis_collection, thresh_b1b2, thresh_b1, thresh_b7):
    def water_flag(img):
        # Apply thresholds to each ratio/band
        b1b2_ratio = ee.Image(img.select("b1b2_ratio"))
        b1b2_sliced = b1b2_ratio.lt(ee.Image.constant(thresh_b1b2))
        b1_sliced = img.select(["red_250m"], ["b1_thresh"]).lt(ee.Image.constant(2027))
        b7_sliced = img.select(["swir"], ["b7_thresh"]).lt(ee.Image.constant(thresh_b7))

        # Combine thresholds (all must pass)
        thresholds = b1b2_sliced.addBands(b1_sliced).addBands(b7_sliced)
        thresholds_count = thresholds.reduce(ee.Reducer.sum())

        # Water flag: at least 3 conditions met
        water_flag = thresholds_count.gte(ee.Image.constant(3))
        return water_flag.copyProperties(img).set(
            "system:time_start", img.get("system:time_start")
        )

    return modis_collection.map(water_flag)
```

Three Threshold Variants: 1. **Main threshold:** `thresh_b1b2=675`, `thresh_b1=2027`, `thresh_b7=675` 2. **Lower bound:** `thresh_b1b2=540`, `thresh_b1=1621.6`, `thresh_b7=540` 3.

Upper bound: thresh_b1b2=810, thresh_b1=2432.4, thresh_b7=810

This provides uncertainty bounds for water detection.

6. Spatial Aggregation Statistics are calculated for each polygon using `reduceRegion`:

```
def calcmean(regiongeom, image):
    mean = image.reduceRegion(
        reducer=ee.Reducer.mean(),
        geometry=regiongeom,
        scale=250
    )
    return ee.Feature(None, mean)

def calcpct(regiongeom, image):
    ptiles = image.reduceRegion(
        reducer=ee.Reducer.percentile([10, 20, 30, 40, 50, 60, 70, 80, 90]),
        geometry=regiongeom,
        scale=250,
    )
    return ee.Feature(None, ptiles)
```

Spatial Aggregation Details: - **Scale:** 250 meters (each pixel is 250m × 250m) - **Distribution:**

All pixel values within the polygon boundary - **For a 1km × 1km polygon:** 16 pixels (4 × 4 grid) -

Statistics calculated: Mean and percentiles (10th, 20th, ..., 90th)

Example for 1km² polygon: - 16 pixels sampled at 250m resolution - Percentiles calculated from the distribution of these 16 pixel values - 50th percentile = median of the 16 values

7. Statistics Processing The pipeline processes statistics in chunks to avoid Earth Engine's 5000 element limit:

```
def process_statistics_chunk(collection, chunk_size=200):
    results = []
    collection_size = collection.size().getInfo()

    for i in range(0, collection_size, chunk_size):
        chunk = collection.toList(chunk_size, i)
        chunk = ee.ImageCollection(chunk)

        # Calculate means and percentiles for this chunk
        bandmean = chunk.map(lambda img: calcmean(regiongeom, img))
        bandpct = chunk.map(lambda img: calcpct(regiongeom, img))

        # Convert to DataFrames and merge
        bandmean_df = fc_to_dataframe(bandmean)
        bandpct_df = fc_to_dataframe(bandpct)
        chunk_df = pd.merge(bandmean_df, bandpct_df, on="index", how="outer")
        results.append(chunk_df)
```

```
return pd.concat(results) if results else pd.DataFrame()
```

Statistics calculated for: 1. Main MODIS collection (all bands) 2. Water detection (main threshold) 3. Water detection (lower bound) 4. Water detection (upper bound)

8. Output Structure The final output CSV contains:

Columns per band: - {band}_mean: Mean value across pixels in polygon - {band}_p10, {band}_p20, ..., {band}_p90: Percentiles

Bands included: - Spectral bands: red_250m, nir_250m, blue, green, swir, nir_500m, band5, band6 - Derived: b1b2_ratio - QA flags: cloud_state, cloud_shadow, ice_flag, snow_flag - Water flags: water_flag (main, lower bound, upper bound variants) - Observation counts: obs250, obs500

Example row structure:

```
index, polygon, red_250m_mean, red_250m_p10, ..., red_250m_p90,
      nir_250m_mean, nir_250m_p10, ..., water_flag_mean, ...
```

Configuration

The MODIS pipeline uses a YAML configuration file:

```
project_id: "ee-ageidv"
output_path: "/content/drive/MyDrive/SAR_rgl/output_2"
shapefile_path: "users/ageidv/your_shapefile"
id_field: "id"
start_date: "2024-01-01"
end_date: "2024-05-31"
max_workers: 48
batch_size: 10000
```

Error Handling

- **Retry mechanism:** Up to 3 retries with exponential backoff
- **Rate limiting:** Delays between batches to avoid EE quota limits
- **Chunked processing:** Handles large collections by splitting into chunks
- **Failed ID tracking:** Records polygon IDs that fail processing

SAR Pipeline

Purpose

The SAR pipeline processes Sentinel-1 Synthetic Aperture Radar (SAR) imagery to detect water using radar backscatter. SAR is advantageous because it can penetrate clouds and operates day/night.

Pipeline Architecture

1. Initialization & Configuration

- Load YAML config
- Initialize Earth Engine
- Setup logging

↓

2. Environment Setup

- Load shapefile FeatureCollection
- Filter by bounding box
- Extract polygon IDs
- Load DEM and HAND datasets

↓

3. For Each Polygon (Parallel Processing)

Get Sentinel-1 collection (ASCENDING/DESCENDING)
Apply terrain corrections
Calculate backscatter statistics
Water detection using thresholds
Calculate statistics (mean + percentiles)

↓

4. Output Generation

- Save results by orbit direction
- Track failed polygon IDs

Detailed Processing Steps

1. Data Collection The pipeline uses Sentinel-1 SAR data, which comes in two orbit directions:

- **ASCENDING:** Satellite moving north to south
- **DESCENDING:** Satellite moving south to north

Example collection loading

```
sar_collection = (  
    ee.ImageCollection("COPERNICUS/S1_GRD")  
    .filterDate(date_range)  
    .filterBounds(regiongeom)  
    .filter(ee.Filter.eq('orbitProperties_pass', direction)) # 'ASCENDING' or 'DESCENDING'  
)
```

Key SAR Properties: - **Resolution:** ~10m (much finer than MODIS) - **Frequency:** C-band (5.4 GHz) - **Polarization:** VV and VH (vertical transmit/vertical receive, vertical transmit/horizontal re-

ceive) - **Temporal frequency:** 6-12 days (depending on location)

2. Terrain Corrections SAR backscatter is affected by terrain. The pipeline applies corrections using DEM (Digital Elevation Model) and HAND (Height Above Nearest Drainage):

```
# Load elevation datasets
dem = ee.Image("MERIT/Hydro/v1_0_1").select("elv").unmask(0)
hand = ee.Image("MERIT/Hydro/v1_0_1").select("hnd").unmask(0)

# Apply terrain corrections using hydrafloods library
corrected = corrections.terrain_correction(
    sar_collection,
    dem=dem,
    hand=hand
)
```

Why terrain correction matters: - Slopes facing the sensor appear brighter (layover) - Slopes facing away appear darker (shadowing) - Corrections normalize these effects for accurate water detection

3. Water Detection Water detection in SAR relies on the fact that smooth water surfaces act like mirrors, reflecting radar signals away from the sensor, resulting in very low backscatter values.

```
def water_detection_sar(sar_image, threshold=-20):
    """
    Detect water using backscatter threshold.
    Water typically has backscatter < -20 dB in VV polarization.
    """
    vv = sar_image.select('VV')
    water = vv.lt(ee.Image.constant(threshold))
    return water.copyProperties(sar_image)
```

Typical thresholds: - **VV polarization:** -20 dB to -18 dB - **VH polarization:** -25 dB to -22 dB

Advantages of SAR for water detection: - Works through clouds - Day/night capability - High spatial resolution (~10m) - Sensitive to surface roughness (smooth water = low backscatter)

4. Spatial Aggregation Similar to MODIS, statistics are calculated using `reduceRegion`:

```
def calcmean(regiongeom, image):
    mean = image.reduceRegion(
        reducer=ee.Reducer.mean(),
        geometry=regiongeom,
        scale=10 # 10m resolution for SAR
    )
    return ee.Feature(None, mean)

def calcpct(regiongeom, image):
    ptiles = image.reduceRegion(
        reducer=ee.Reducer.percentile([10, 20, 30, 40, 50, 60, 70, 80, 90]),
```

```

        geometry=regiongeom,
        scale=10,
    )
    return ee.Feature(None, ptiles)

```

Key Difference from MODIS: - **Scale:** 10 meters (vs 250m for MODIS) - **For a 1km x 1km polygon:** 10,000 pixels (100 x 100 grid) - Much finer spatial sampling provides more detailed statistics

Example for 1km² polygon: - 10,000 pixels sampled at 10m resolution - Percentiles calculated from distribution of 10,000 pixel values - More robust statistics due to larger sample size

5. Processing by Orbit Direction SAR data is processed separately for ascending and descending orbits:

```

for direction in ['ASCENDING', 'DESCENDING']:
    # Filter collection by orbit direction
    sar_collection = (
        ee.ImageCollection("COPERNICUS/S1_GRD")
        .filterDate(date_range)
        .filterBounds(regiongeom)
        .filter(ee.Filter.eq('orbitProperties_pass', direction))
    )

    # Process and calculate statistics
    stats = process_sar_collection(sar_collection, regiongeom)

    # Save with direction identifier
    save_results(stats, f"sar_results_{direction}.csv")

```

Why separate processing: - Different viewing geometries can affect backscatter - Allows comparison between orbit directions - Some areas may have better coverage in one direction

6. Output Structure SAR outputs are similar to MODIS but include:

Columns per band: - VV_mean, VH_mean: Mean backscatter (dB) - VV_p10, ..., VV_p90: Percentiles for VV - VH_p10, ..., VH_p90: Percentiles for VH - water_flag_mean: Mean water detection (0-1) - water_flag_p10, ..., water_flag_p90: Water detection percentiles

Additional metadata: - orbitProperties_pass: ASCENDING or DESCENDING - relativeOrbitNumber_start
Orbit number - system:time_start: Image acquisition time

Configuration

SAR pipeline configuration:

```

project_id: "ee-ageidv"
output_path: "/content/drive/MyDrive/SAR_rgl/output"
shapefile_path: "users/ageidv/your_shapefile"
id_field: "id"

```

```
start_date: "2024-01-01"
end_date: "2024-05-31"
max_workers: 15
group: "A" # For batch processing (A, B, C, D, E)
```

Output Integration

Common Keys for Integration

Both pipelines produce outputs that can be integrated using:

1. **Polygon ID:** Both use the same polygon field from the shapefile
2. **Temporal Index:** Both include `system:time_start` or date information
3. **Spatial Reference:** Both reference the same Earth Engine FeatureCollection

Integration Strategy

Step 1: Load Both Datasets

```
import pandas as pd

# Load MODIS results
modis_df = pd.read_csv('modis_results_20240101_20240531.csv')

# Load SAR results (both directions)
sar_asc_df = pd.read_csv('sar_20240101_20240531_results_ASCENDING.csv')
sar_desc_df = pd.read_csv('sar_20240101_20240531_results_DESCENDING.csv')

# Combine SAR directions
sar_df = pd.concat([sar_asc_df, sar_desc_df], ignore_index=True)
```

Step 2: Standardize Temporal Index Both datasets need a common date/time field:

```
# MODIS: Extract date from system:time_start or index
modis_df['date'] = pd.to_datetime(modis_df['system:time_start'], unit='ms')

# SAR: Extract date from system:time_start
sar_df['date'] = pd.to_datetime(sar_df['system:time_start'], unit='ms')

# Round to nearest day for matching
modis_df['date_day'] = modis_df['date'].dt.date
sar_df['date_day'] = sar_df['date'].dt.date
```

Step 3: Merge on Polygon and Date

```
# Merge MODIS and SAR on polygon ID and date
merged_df = pd.merge(
    modis_df,
```

```

sar_df,
on=['polygon', 'date_day'],
how='outer', # Keep all observations
suffixes=('_modis', '_sar')
)

```

Step 4: Create Composite Water Detection Combine MODIS and SAR water detections for more robust results:

```

def composite_water_detection(row):
    """
    Combine MODIS and SAR water detections.
    Water if either sensor detects it, or both agree.
    """
    modis_water = row.get('water_flag_mean_modis', 0)
    sar_water = row.get('water_flag_mean_sar', 0)

    # If both sensors agree, high confidence
    if modis_water > 0.5 and sar_water > 0.5:
        return 1.0, 'high_confidence'
    # If either detects water, medium confidence
    elif modis_water > 0.5 or sar_water > 0.5:
        return 0.75, 'medium_confidence'
    # If both indicate no water, no water
    else:
        return 0.0, 'no_water'

merged_df[['composite_water', 'confidence']] = merged_df.apply(
    lambda row: pd.Series(composite_water_detection(row)),
    axis=1
)

```

Step 5: Handle Temporal Mismatches MODIS has daily coverage, SAR has 6-12 day frequency. Use temporal interpolation or nearest neighbor:

```

# Option 1: Forward fill SAR values to daily
sar_daily = (
    sar_df.set_index(['polygon', 'date_day'])
    .reindex(
        pd.MultiIndex.from_product([
            sar_df['polygon'].unique(),
            pd.date_range(sar_df['date_day'].min(), sar_df['date_day'].max())
        ], names=['polygon', 'date_day'])
    )
    .groupby('polygon').ffill()
    .reset_index()
)

```

```

# Option 2: Nearest neighbor matching
from scipy.spatial.distance import cdist

def match_nearest_date(modis_date, sar_dates):
    """Find nearest SAR date to MODIS date"""
    distances = abs((pd.to_datetime(sar_dates) - pd.to_datetime(modis_date)).days)
    return sar_dates[distances.idxmin()]

```

Step 6: Create Integrated Dataset

```

# Final integrated dataset
integrated_df = merged_df[[
    'polygon',
    'date_day',
    # MODIS features
    'red_250m_mean_modis',
    'nir_250m_mean_modis',
    'b1b2_ratio_mean_modis',
    'water_flag_mean_modis',
    # SAR features
    'VV_mean_sar',
    'VH_mean_sar',
    'water_flag_mean_sar',
    # Composite
    'composite_water',
    'confidence',
    # QA flags
    'cloud_state_mean_modis',
    'cloud_shadow_mean_modis',
]].copy()

# Save integrated dataset
integrated_df.to_csv('integrated_modis_sar_results.csv', index=False)

```

Integration Benefits

1. **Complementary Coverage:** MODIS fills temporal gaps in SAR, SAR fills cloud gaps in MODIS
2. **Validation:** Cross-validate water detections between sensors
3. **Uncertainty Quantification:** Use agreement between sensors as confidence metric
4. **Robust Detection:** Water detected by both sensors is highly reliable

Example Integration Workflow

```

"""
Complete integration workflow
"""

```

```

import pandas as pd
import numpy as np
from datetime import datetime

# 1. Load datasets
modis = pd.read_csv('modis_results_20240101_20240531.csv')
sar_asc = pd.read_csv('sar_20240101_20240531_ASCENDING.csv')
sar_desc = pd.read_csv('sar_20240101_20240531_DESCENDING.csv')

# 2. Standardize
modis['date'] = pd.to_datetime(modis['system:time_start'], unit='ms').dt.date
sar_asc['date'] = pd.to_datetime(sar_asc['system:time_start'], unit='ms').dt.date
sar_desc['date'] = pd.to_datetime(sar_desc['system:time_start'], unit='ms').dt.date

# 3. Combine SAR
sar = pd.concat([sar_asc, sar_desc]).reset_index(drop=True)

# 4. Create daily grid
all_polygons = sorted(set(modis['polygon'].unique()) | set(sar['polygon'].unique()))
all_dates = pd.date_range(
    min(modis['date'].min(), sar['date'].min()),
    max(modis['date'].max(), sar['date'].max())
).date

# 5. Merge
modis_pivot = modis.set_index(['polygon', 'date'])
sar_pivot = sar.set_index(['polygon', 'date'])

merged = pd.merge(
    modis_pivot,
    sar_pivot,
    left_index=True,
    right_index=True,
    how='outer',
    suffixes=('_modis', '_sar')
).reset_index()

# 6. Create composite water flag
merged['composite_water'] = (
    (merged['water_flag_mean_modis'].fillna(0) > 0.5) |
    (merged['water_flag_mean_sar'].fillna(0) > 0.5)
).astype(float)

merged['both_sensors'] = (
    (merged['water_flag_mean_modis'].fillna(0) > 0.5) &
    (merged['water_flag_mean_sar'].fillna(0) > 0.5)
).astype(float)

```

```
# 7. Save
merged.to_csv('integrated_modis_sar_flood_detection.csv', index=False)
print(f"Integrated dataset: {len(merged)} rows, {len(merged.columns)} columns")
```

Common Components

1. Earth Engine Asset

Both pipelines use the same shapefile:

```
shapefile = ee.FeatureCollection("users/ageidv/your_shapefile")
```

Shapefile Structure: - Must have a unique ID field (specified in config as `id_field`) - Polygons should be in a regular grid (e.g., 1 km × 1 km) - Same coordinate reference system (typically WGS84)

2. Bounding Box Filtering

Both pipelines filter polygons by bounding box:

MODIS:

```
BOUNDS = {
    "minx": -51.65, # West
    "miny": -28.45, # South
    "maxx": -51.25, # East
    "maxy": -28.05  # North
}
```

SAR:

```
bbox = ee.Geometry.Rectangle([
    -53.7, # minx
    -30.2, # miny
    -52.35, # maxx
    -29.61 # maxy
])
```

Note: These bounding boxes may differ based on study area requirements.

3. Elevation Datasets

Both pipelines load elevation data (though MODIS may not use it directly):

```
dem = ee.Image("MERIT/Hydro/v1_0_1").select("elv").unmask(0)
hand = ee.Image("MERIT/Hydro/v1_0_1").select("hnd").unmask(0)
```

- **DEM:** Digital Elevation Model (terrain height)
- **HAND:** Height Above Nearest Drainage (flood modeling)

4. Parallel Processing

Both use ThreadPoolExecutor for parallel polygon processing:

```
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    futures = {
        executor.submit(process_polygon, polygon_id): polygon_id
        for polygon_id in polygon_ids
    }

    for future in tqdm(futures):
        result = future.result()
        results.append(result)
```

5. Error Handling

Both implement retry mechanisms:

```
def process_with_retry(polygon_id, max_retries=3):
    for attempt in range(max_retries):
        try:
            return process_polygon(polygon_id)
        except ee.EEException as e:
            if "rate" in str(e).lower():
                time.sleep(30 * (attempt + 1)) # Exponential backoff
            else:
                raise
    return None
```

6. Logging

Both use comprehensive logging:

```
def setup_logger(log_file_path):
    logging.root.setLevel(logging.INFO)

    file_handler = logging.FileHandler(log_file_path)
    stream_handler = logging.StreamHandler(sys.stdout)

    formatter = logging.Formatter(
        '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
    )

    file_handler.setFormatter(formatter)
    stream_handler.setFormatter(formatter)

    logger = logging.getLogger()
    logger.addHandler(file_handler)
    logger.addHandler(stream_handler)
```

`return` logger

Summary

MODIS Pipeline Highlights

- **Optical sensor:** Uses visible and near-infrared bands
- **Daily temporal resolution:** High frequency monitoring
- **250m spatial resolution:** After pan-sharpening
- **Cloud-sensitive:** Requires clear skies
- **Spectral water detection:** Based on reflectance ratios
- **Output:** Mean and percentiles for all bands + water flags

SAR Pipeline Highlights

- **Radar sensor:** Uses microwave backscatter
- **6-12 day temporal resolution:** Lower frequency but all-weather
- **10m spatial resolution:** Much finer than MODIS
- **Cloud-penetrating:** Works in all weather conditions
- **Backscatter water detection:** Based on low radar return
- **Output:** Mean and percentiles for VV/VH + water flags

Integration Benefits

1. **Temporal complementarity:** MODIS fills SAR gaps, SAR fills MODIS cloud gaps
2. **Spatial complementarity:** SAR provides detail, MODIS provides context
3. **Validation:** Cross-sensor agreement increases confidence
4. **Robustness:** Multiple sensors reduce false positives/negatives

Best Practices

1. **Run both pipelines** for the same time period and polygons
 2. **Standardize outputs** before integration (date formats, column names)
 3. **Handle temporal mismatches** (SAR less frequent than MODIS)
 4. **Use composite flags** that combine both sensors
 5. **Track confidence** based on sensor agreement
 6. **Validate** integrated results against ground truth when available
-

Appendix: Key Functions Reference

MODIS Functions

Function	Purpose
dfo_bands_gq()	Rename GQ (250m) bands
dfo_bands_ga()	Rename GA (500m) bands
join_collections()	Join GQ and GA by timestamp
pan_sharpen()	Upscale 500m to 250m
b1b2_ratio()	Calculate NIR/Red ratio
add_qa_bands()	Extract QA flags
water_detection()	Detect water using thresholds
calcmean()	Calculate mean over polygon
calcpct()	Calculate percentiles over polygon

SAR Functions

Function	Purpose
terrain_correction()	Correct for terrain effects
water_detection_sar()	Detect water using backscatter
calcmean()	Calculate mean over polygon
calcpct()	Calculate percentiles over polygon
process_single_polygon()	Process one polygon (parallel)

Documentation generated: 2024 Last updated: Based on MODIS_RioGrande_v1.ipynb and SAR_RioGrande_v2.ipynb